INDEX

Sr. No.	Description of Topic	Page No.
1.	Introduction	1
2.	Scope of Work	2
3.	An Existing System	3
4.	Need of Proposed System	4
5.	Feasibility Study	5
6.	System Features	6
7.	Hardware and Software	7
8.	Data Flow Diagram	8
9.	Database Layout	12
10.	Design of Input Screens	14
11.	Design of Output Screens & Reports	20
12.	Testing Procedures & Implementation Phases	25
13.	Limitations	27
14.	Conclusion	28

1. INTRODUCTION

Platinum Kicks - Shoes Ecommerce is a desktop-based footwear shopping and administration application developed using Visual Basic .NET Windows Forms in Microsoft Visual Studio 2010. It uses a local Microsoft SQL Server database stored in an MDF file named PlatinumKicks.mdf.

The application provides two role-based areas: a customer-facing area for account registration, login, browsing footwear products, maintaining a cart and placing an order; and an administration area for maintaining product records and viewing customer orders. The system stores product information and images, user accounts, order headers, and order line items in relational database tables.

The project demonstrates the integration of a Windows desktop interface with database operations, form navigation, product image handling, cart management, checkout processing and stock updates. Payment options are simulated for demonstration and do not connect to an external payment gateway.

1.1 Project Objectives

- To provide a clear desktop interface for browsing and selecting footwear products.
- To support user registration and role-aware login for customer and administrator accounts.
- To enable administrators to add, update and delete product records and maintain stock information.
- To maintain cart items during a shopping session and calculate the order total.
- To capture delivery details and a selected payment method during checkout.
- To store order and order-item records and update product stock when an order is placed.
- To centralize database operations through a dedicated DatabaseHelper component.

1.2 Project Identity

Item	Description
Project title	Platinum Kicks - Shoes Ecommerce
Application type	Windows Forms desktop application
Programming language	Visual Basic .NET (VB.NET)
Development environment	Microsoft Visual Studio 2010
Database platform	Microsoft SQL Server Express
Database file	PlatinumKicks.mdf
Primary roles	User / Customer and Admin

2. SCOPE OF WORK

The scope of Platinum Kicks is limited to the core activities required for a small footwear e-commerce simulation. It combines customer shopping operations with administrator-side product and order administration.

2.1 Customer Scope

- Create an account using full name, username, email and password.
- Log in using registered credentials.
- Browse available footwear and view product information, prices, stock and product images.
- Select a product and quantity and add it to the in-memory cart.
- Review cart items and the calculated total; remove items where supported by the cart interface.
- Enter delivery details and select Cash on Delivery, UPI or Card as a simulated payment method.
- Place an order, save the order and its line items, and trigger stock reduction.

2.2 Administrator Scope

- Log in to the administrator area.
- Add a product with its name, brand, description, price, stock and image.
- View the product list and update or delete product records.
- View customer orders and the available order information.

2.3 Technical Scope

The application is built as a local Windows desktop program. SQL Server stores persistent records in four tables. The cart is maintained in application memory using CartManager and CartItem rather than in a separate SQL Server cart table. Product images are stored as binary data in the Products table.

2.4 Out of Scope

- Real online payment authorization or payment-gateway integration.
- Public web hosting, customer access through a browser, or cloud-based deployment.
- Courier integration, shipment tracking and automated delivery notifications.
- A separate persistent cart table or a separate payment transaction table.

3. AN EXISTING SYSTEM

In a basic manual footwear-selling workflow, product details, stock levels and customer orders may be maintained through paper records, spreadsheets or separate records. Such a process can require repeated data entry and manual cross-checking between product availability, customer details and order information.

Where product information and orders are maintained separately, it can be difficult to maintain consistent stock quantities and retrieve order details quickly. A manual process may also make it harder to keep product descriptions, prices and images organized in a single catalogue.

3.1 Common Challenges in a Manual Workflow

- Product information may be duplicated or become inconsistent across records.
- Stock updates may be delayed after a sale or order.
- Finding a particular product or order may require manual searching.
- Order details and line items may not be linked systematically.
- Administrative product maintenance may require repeated editing of separate records.

These points describe the general context that the proposed application addresses; they are not claims about a particular commercial system.

4. NEED OF PROPOSED SYSTEM

Platinum Kicks addresses the need for a consolidated desktop workflow that connects product maintenance, customer shopping and order recording through a relational database. The system reduces the need to maintain product, user and order information in unrelated records.

4.1 Rationale

- Centralized storage: user, product and order information is maintained in SQL Server.
- Consistent product catalogue: administrators can add, update and remove product records through dedicated forms.
- Integrated shopping flow: product browsing, cart handling and checkout are connected within the same application.
- Order traceability: an order header is stored separately from its individual order items.
- Stock control: checkout updates product stock and checks that sufficient quantity is available.
- Usable interface: separate Admin and User forms make the available actions clearer for each role.

4.2 Expected Benefits

The proposed system is intended to improve the organization of footwear data, reduce repetitive manual handling, make product maintenance more direct and demonstrate an end-to-end order workflow in a desktop environment.

5. FEASIBILITY STUDY

A feasibility study considers whether the project can be implemented with the available technical resources, development time and operating environment.

5.1 Technical Feasibility

The project uses established technologies available in the development environment: VB.NET Windows Forms, Visual Studio 2010, SQL Server Express and an MDF database. The database helper provides a centralized location for database operations, while forms handle user interaction. The completed application has been run and tested on the development/home PC and the college PC.

5.2 Economic Feasibility

The project is suitable for an academic demonstration because it uses a local database and desktop development environment rather than requiring a paid cloud service or a commercial payment gateway. Costs may include the computer hardware and any licensing or installation requirements applicable to the institution.

5.3 Operational Feasibility

The application separates customer operations from administrator operations. Customers use registration, login, shop and cart forms, while the administrator uses a dashboard and dedicated product/order forms. The interface is designed for direct use by a person operating a Windows desktop.

5.4 Schedule Feasibility

The work was completed within the academic project period specified in the college template. Development included database design, form integration, conversion of database operations to SQL Server, debugging, functional testing and preparation of documentation.

5.5 Conclusion of Feasibility

Within the scope of an academic desktop application, the project is technically and operationally feasible. It demonstrates the required database-backed workflows without depending on an external online infrastructure.

6. SYSTEM FEATURES

6.1 User Module

Feature	Description
Landing screen	Provides entry points to login, registration and exit.
Registration	Captures full name, username, email and password for a new account.
Login	Validates the supplied credentials and determines the account role.
Browse products	Displays product records, including product name, brand, description, price, stock and image.
Product selection	Updates the selected product information and allows a quantity to be chosen.
Cart management	Stores selected products in memory, combines quantities for repeated selections and presents cart totals.
Checkout	Collects delivery details and a payment-method selection.
Order placement	Creates an order and its order items, checks stock and reduces stock in the database.

6.2 Administrator Module

Feature	Description
Admin dashboard	Provides navigation to product creation, product management and order viewing.
Add product	Stores product name, brand, description, price, stock and selected product image.
Manage products	Displays product records and provides update and delete operations.
View orders	Displays customer order information retrieved from the database.
Stock maintenance	Product stock is maintained in the Products table and adjusted during successful order placement.

6.3 Shared Components

Component	Responsibility
DatabaseHelper	Centralizes SQL Server connection and database methods for registration, login, product CRUD, order placement, stock reduction and order retrieval.
Session	Maintains the currently logged-in user's ID, username and role.
CartManager	Maintains the active cart in application memory.
CartItem	Represents a product selected for the cart, including product ID, name, price, quantity and total.
BaseForm	Provides shared form-level appearance and behavior.

7. HARDWARE AND SOFTWARE

7.1 Software Requirements

Software / Component	Purpose
Windows operating system	Runs the desktop application and development tools.
Microsoft Visual Studio 2010	Development and debugging environment.
Visual Basic .NET Windows Forms	User-interface and application programming framework.
Microsoft SQL Server Express	Relational database engine for persistent application data.
PlatinumKicks.mdf	Service-based database file containing the application tables.
System.Data.SqlClient	ADO.NET provider used by the application to communicate with SQL Server.

7.2 Hardware Requirements

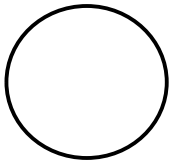
Component	Practical requirement
Computer	Windows-compatible desktop or laptop.
Processor	A processor supported by the selected Windows/Visual Studio installation.
Memory	Sufficient RAM to run Windows, Visual Studio and SQL Server Express concurrently.
Storage	Enough free disk space for the IDE, database file, project and product images.
Display	A display capable of presenting the Windows Forms interface comfortably.

The table describes practical categories rather than asserting a specific minimum hardware configuration, because the exact PC specifications were not supplied.

8. DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) represents the movement of information between external entities, processes and data stores. The context-level diagram shows the system boundary. The two zero-level diagrams expand the main customer and administrator processes.

8.1 DFD Notation

Description	Symbols	Functions
External Entity		A rectangle represents an external entity.
Process		The activities within a system that uses or generate data.
Flow		A connector that shows relationships between the representative shapes.
Data Store		Electronic data storage

8.2 Context-Level DFD



Figure 8.1 — Context-Level DFD

8.3 Zero-Level DFD — User Side

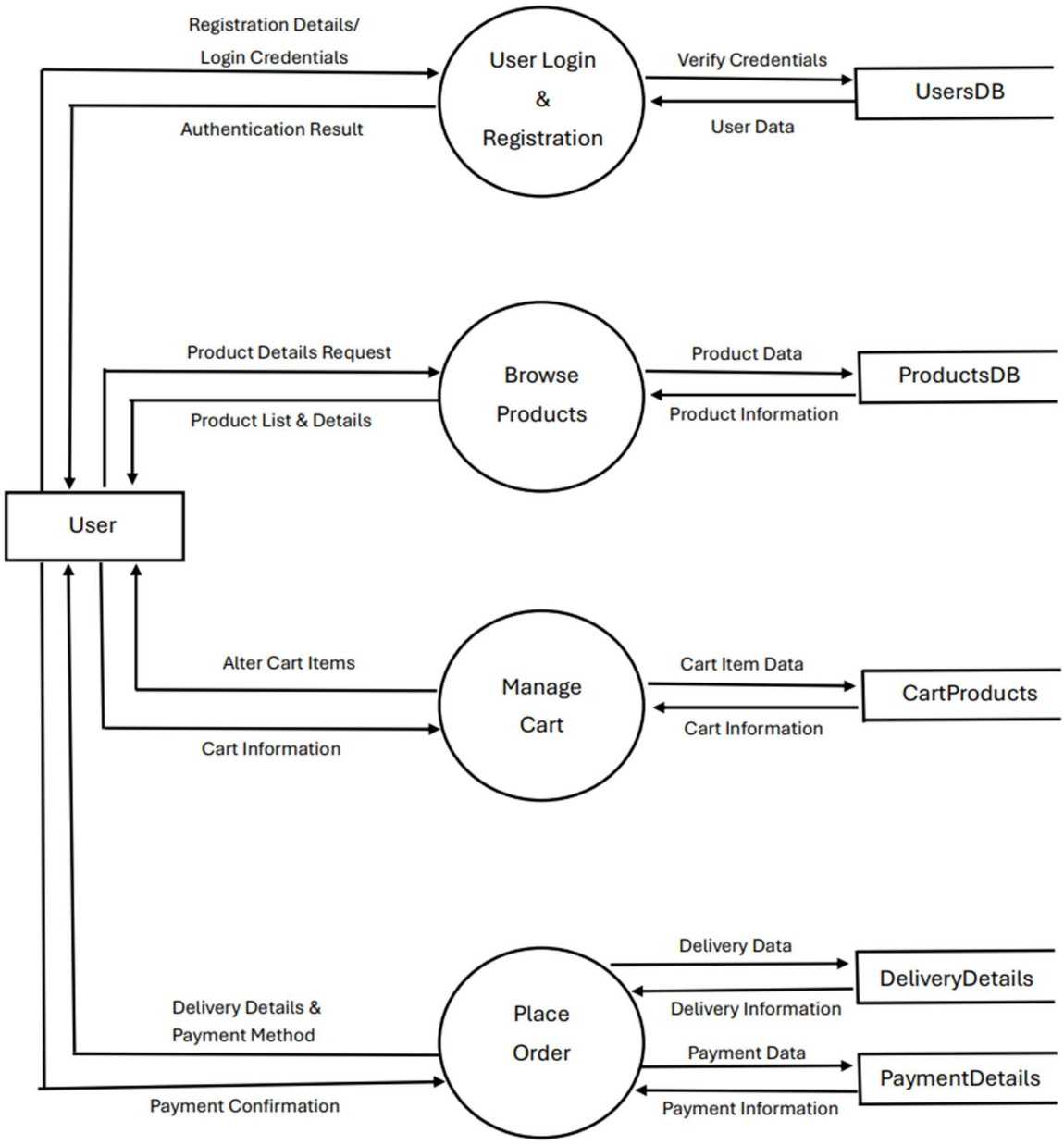


Figure 8.2 — Zero-Level DFD (User Side)

8.4 Zero-Level DFD — Admin Side

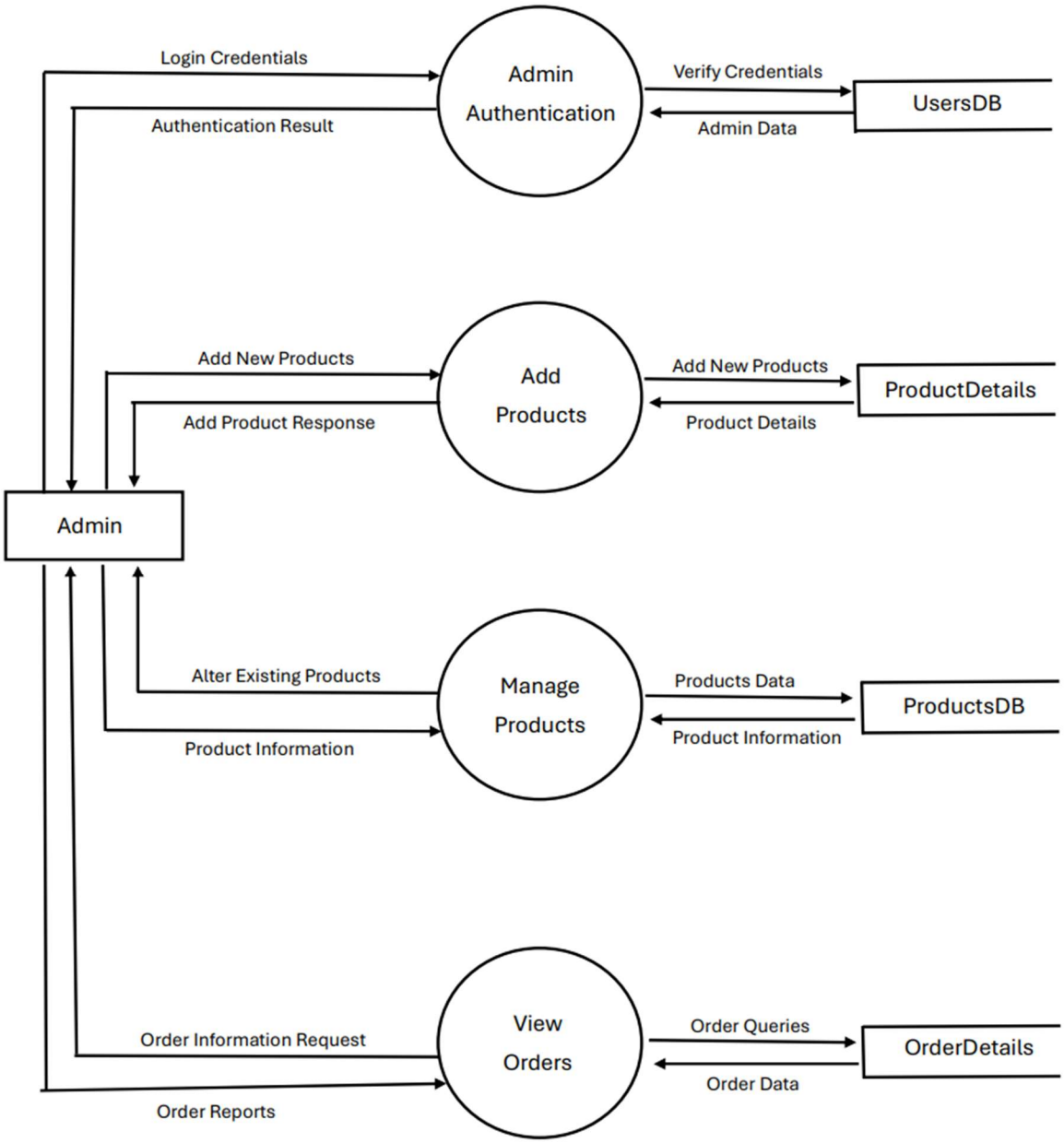


Figure 8.3 — Zero-Level DFD (Admin Side)

9. DATABASE LAYOUT

The PlatinumKicks database is stored in PlatinumKicks.mdf and contains four persistent tables: users, products, orders and order_items. The cart is not a database table; it is maintained temporarily in memory by the application.

9.1 Table: users

Field	SQL Server type	Key / constraint	Purpose
user_id	INT IDENTITY(1,1)	Primary Key	Unique identifier for each account.
full_name	VARCHAR(100)	NOT NULL	Account holder's name.
username	VARCHAR(50)	UNIQUE, NOT NULL	Login username.
email	VARCHAR(100)	UNIQUE, NOT NULL	Email address.
password	VARCHAR(255)	NOT NULL	Credential value used by the application's login flow.
role	VARCHAR(20)	Default 'User'	Role used to distinguish Admin and User.
created_at	DATETIME	Default GETDATE()	Account creation timestamp.

9.2 Table: products

Field	SQL Server type	Key / constraint	Purpose
product_id	INT IDENTITY(1,1)	Primary Key	Unique product identifier.
name	VARCHAR(100)	NOT NULL	Product name.
brand	VARCHAR(50)	NOT NULL	Manufacturer/brand.
description	VARCHAR(MAX)	Nullable	Product description.
price	DECIMAL(10,2)	NOT NULL	Selling price.
stock	INT	NOT NULL	Available quantity.
image	VARBINARY(MAX)	Nullable	Product image stored as binary data.

9.3 Table: orders

Field	SQL Server type	Key / constraint	Purpose
order_id	INT IDENTITY(1,1)	Primary Key	Unique order identifier.
user_id	INT	FK → users.user_id	Customer who placed the order.
address	VARCHAR(255)	NOT NULL	Delivery address.
city	VARCHAR(100)	NOT NULL	Delivery city.
state	VARCHAR(100)	NOT NULL	Delivery state.
pincode	VARCHAR(10)	NOT NULL	Postal PIN code.
payment_method	VARCHAR(30)	NOT NULL	Selected simulated method.
order_date	DATETIME	Default GETDATE()	Order creation date/time.
total_amount	DECIMAL(10,2)	NOT NULL	Total order amount.
status	VARCHAR(30)	Default 'Pending'	Order status.

9.4 Table: order_items

Field	SQL Server type	Key / constraint	Purpose
order_item_id	INT IDENTITY(1,1)	Primary Key	Unique order-line identifier.
order_id	INT	FK → orders.order_id	Parent order.
product_id	INT	FK → products.product_id	Referenced product.
quantity	INT	NOT NULL	Quantity ordered.
price	DECIMAL(10,2)	NOT NULL	Unit price recorded for the line item.

9.5 Relationships

- users (1) → orders (many): one account may be associated with multiple orders.
- orders (1) → order_items (many): an order can contain multiple product lines.
- products (1) → order_items (many): a product can appear in multiple order lines.

Referential integrity is enforced through foreign keys. A product referenced by order_items cannot be deleted unless the configured relationship permits it; the final behavior depends on the database's current foreign-key definition.

9.6 Logical Relationship Summary

Parent table	Parent key	Child table	Child key
users	user_id	orders	user_id
orders	order_id	order_items	order_id
products	product_id	order_items	product_id

10. DESIGN OF INPUT SCREENS

The input screens accept account details, product information, product selections, quantities and checkout details. The screenshots below are taken from the completed PlatinumKicks application.

Landing screen

Entry point for Login, Register and Exit.

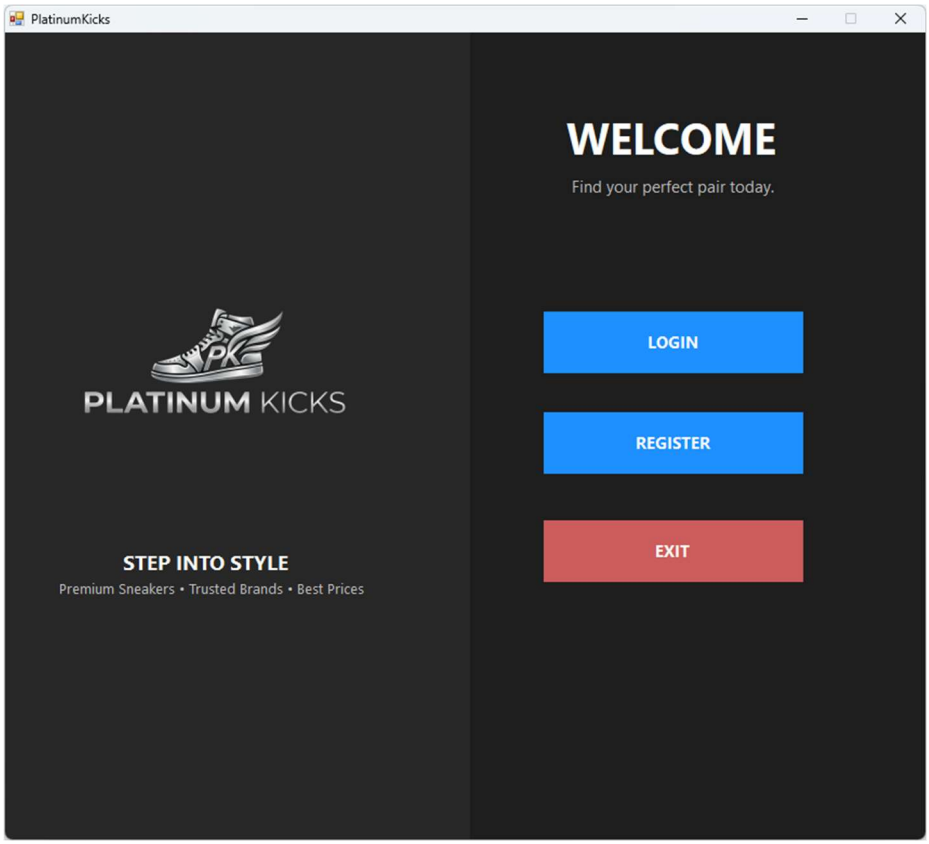


Figure 10.1 — Landing screen (frmLanding)

Login screen

Accepts username and password and starts role-based navigation after authentication.

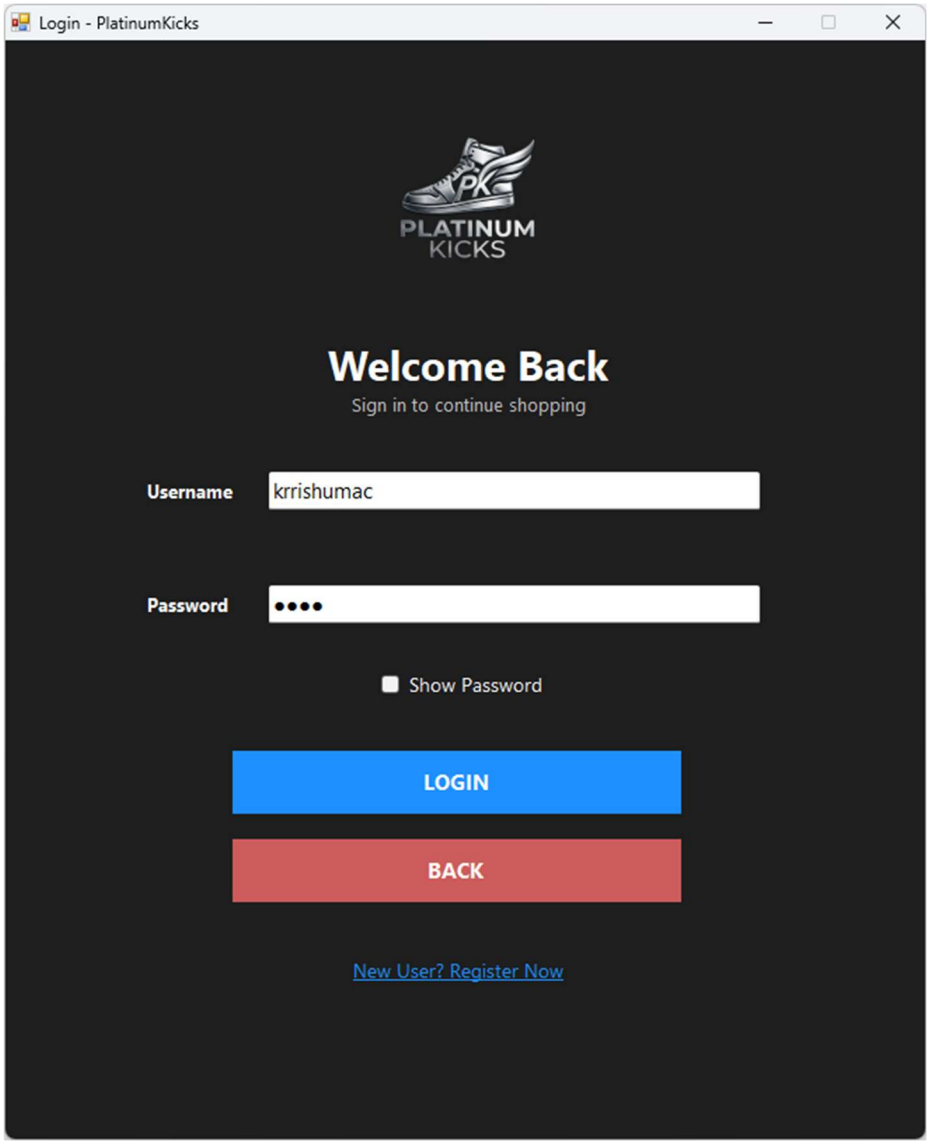


Figure 10.2 — Login screen (frmLogin)

Registration screen

Accepts full name, username, email, password and password confirmation.

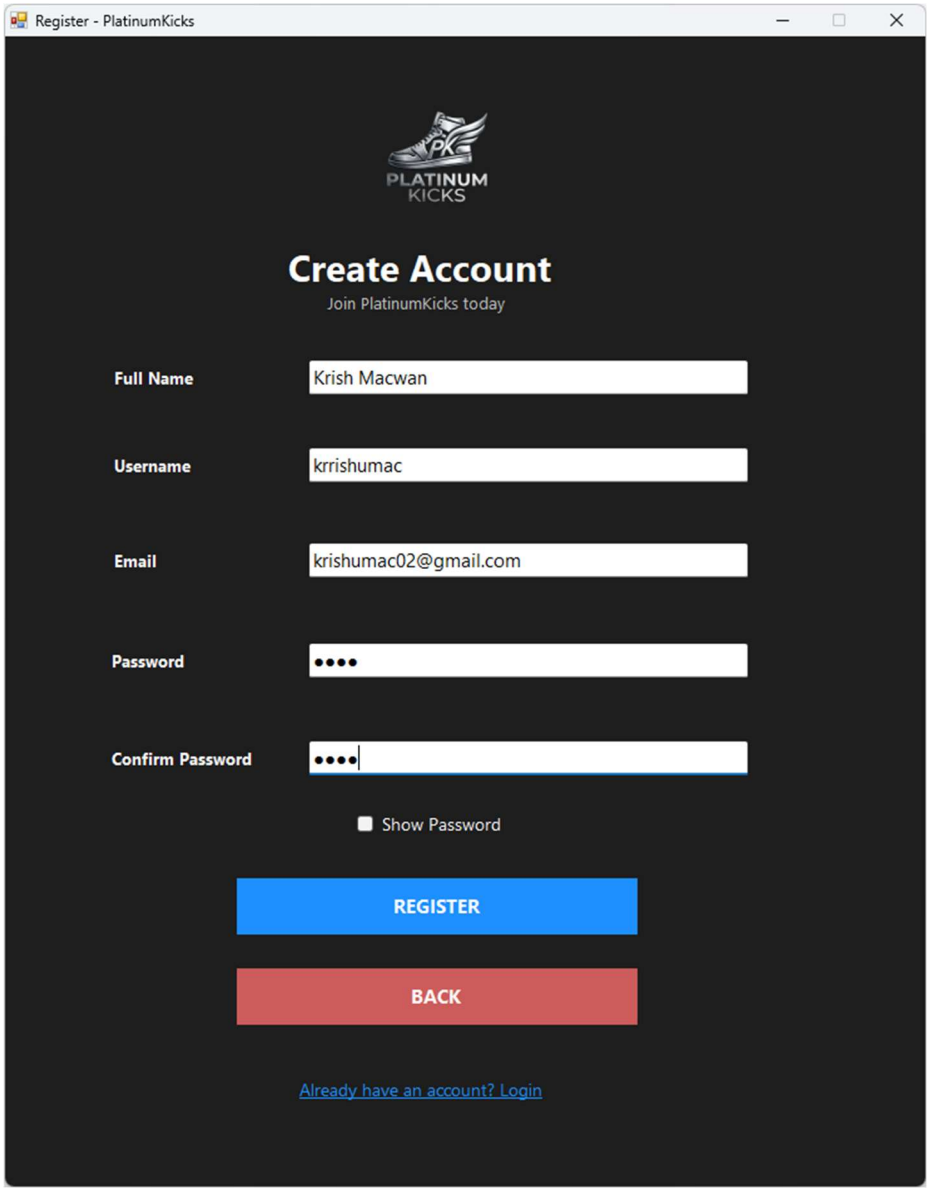


Figure 10.3 — Registration screen (frmRegister)

Add Product screen

Accepts product name, brand, description, price, stock and a product image.

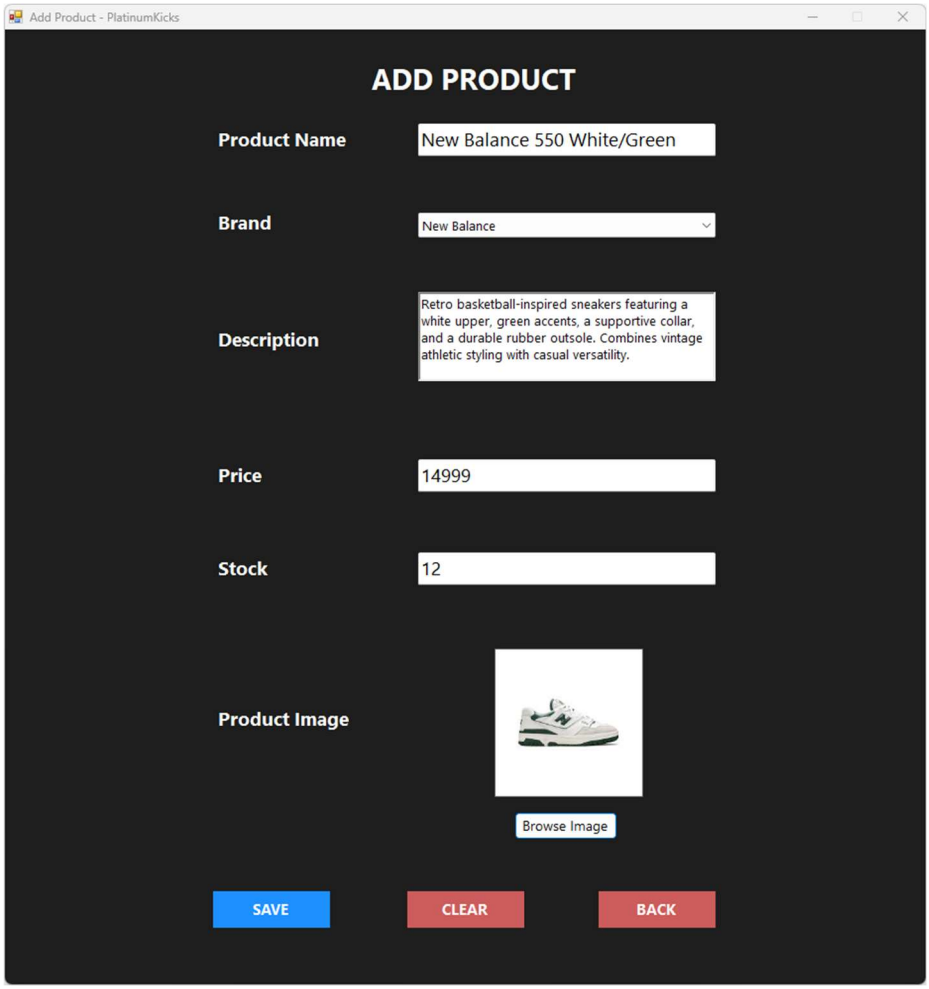


Figure 10.4— Add Product screen (frmAddProduct)

Shop screen

Supports selecting a product and quantity and adding the product to the cart.

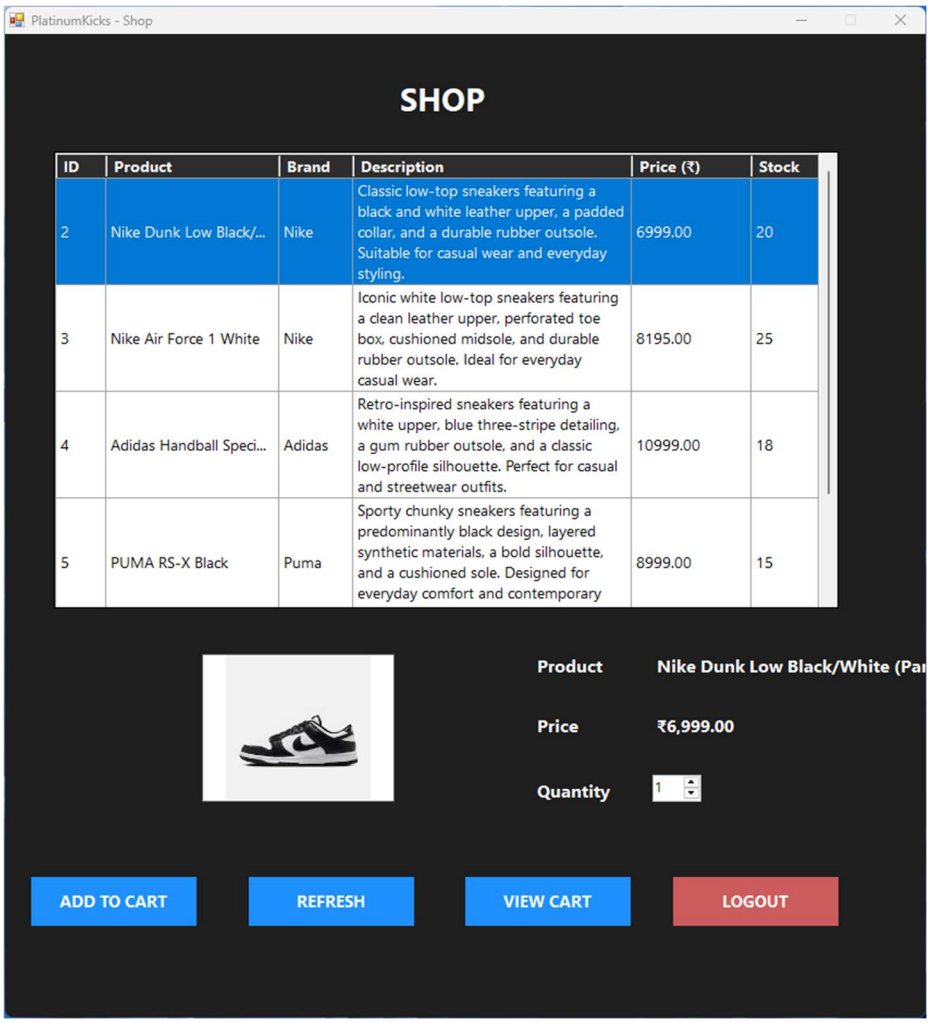


Figure 10.5— Shop screen (frmShop)

Cart and Checkout screen

Displays cart items and total, accepts delivery details and payment method, and submits checkout.

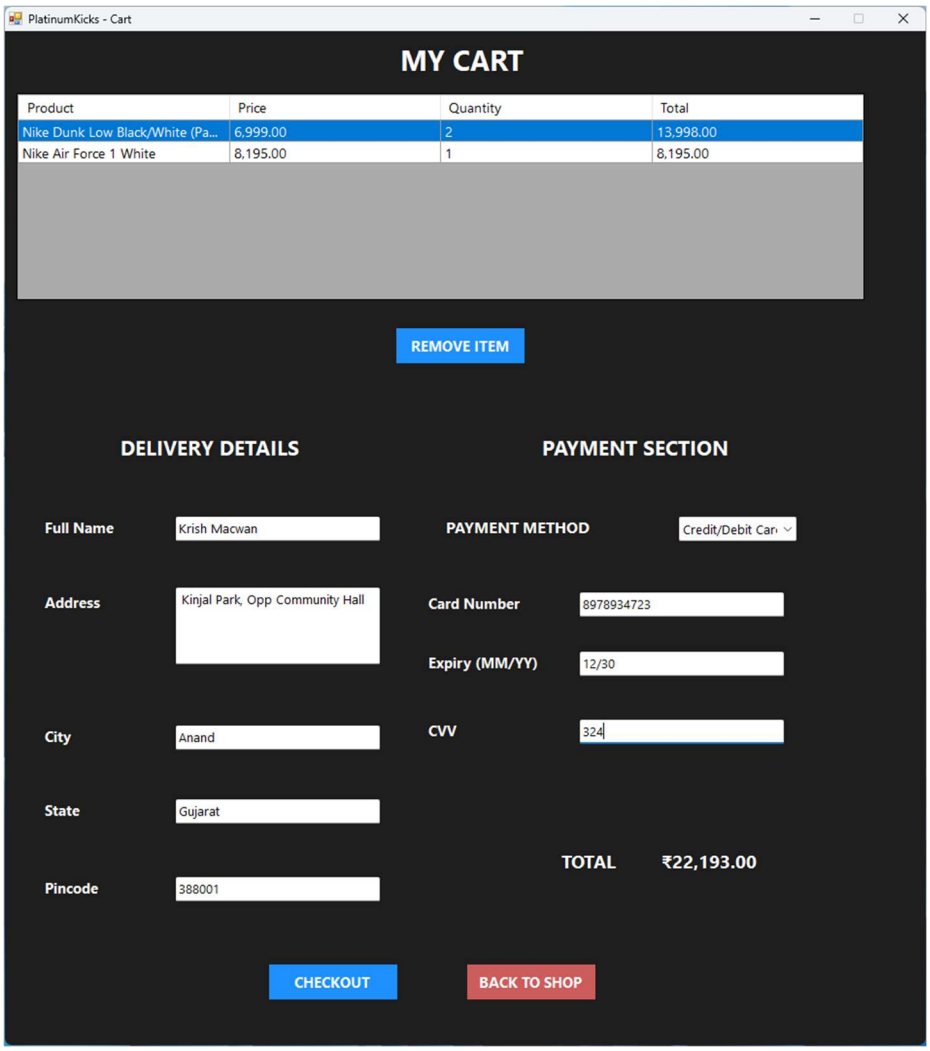


Figure 10.6— Cart and Checkout screen (frmCart)

11. DESIGN OF OUTPUT SCREENS & REPORTS

Output screens present database records and operation results to the customer or administrator. The project does not use a separate report-generation engine; the main administrative output is the product list and order list displayed in Windows Forms.

Admin Dashboard

Provides navigation to Add Product, Manage Products and View Orders.

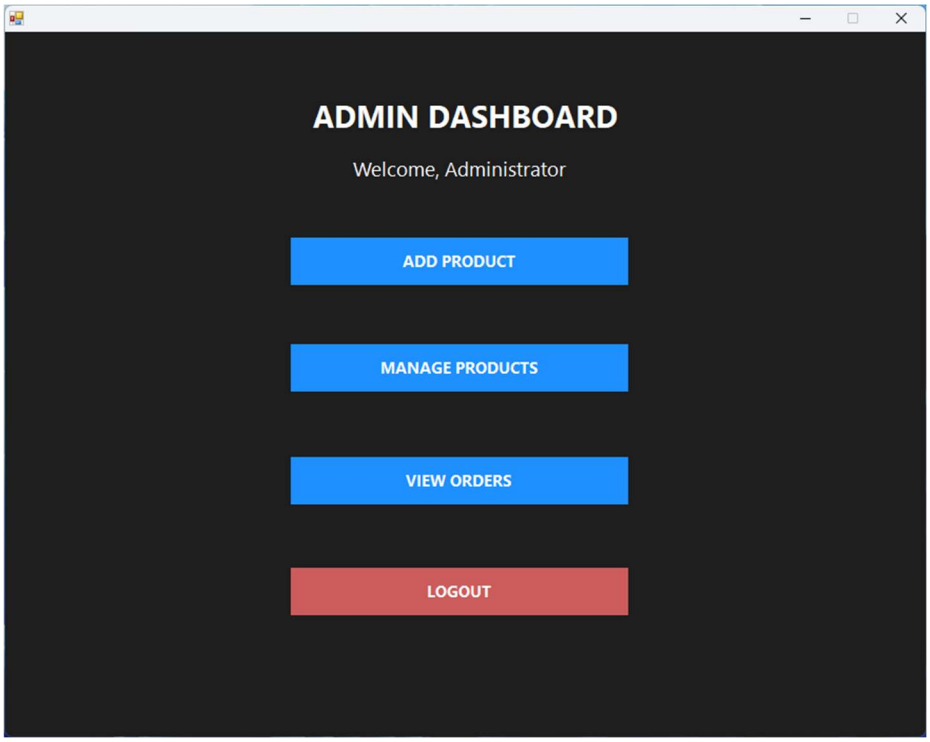


Figure 11.1— Admin Dashboard (frmAdminDashboard)

Manage Products

Displays the product catalogue and supports selecting records for update or deletion.

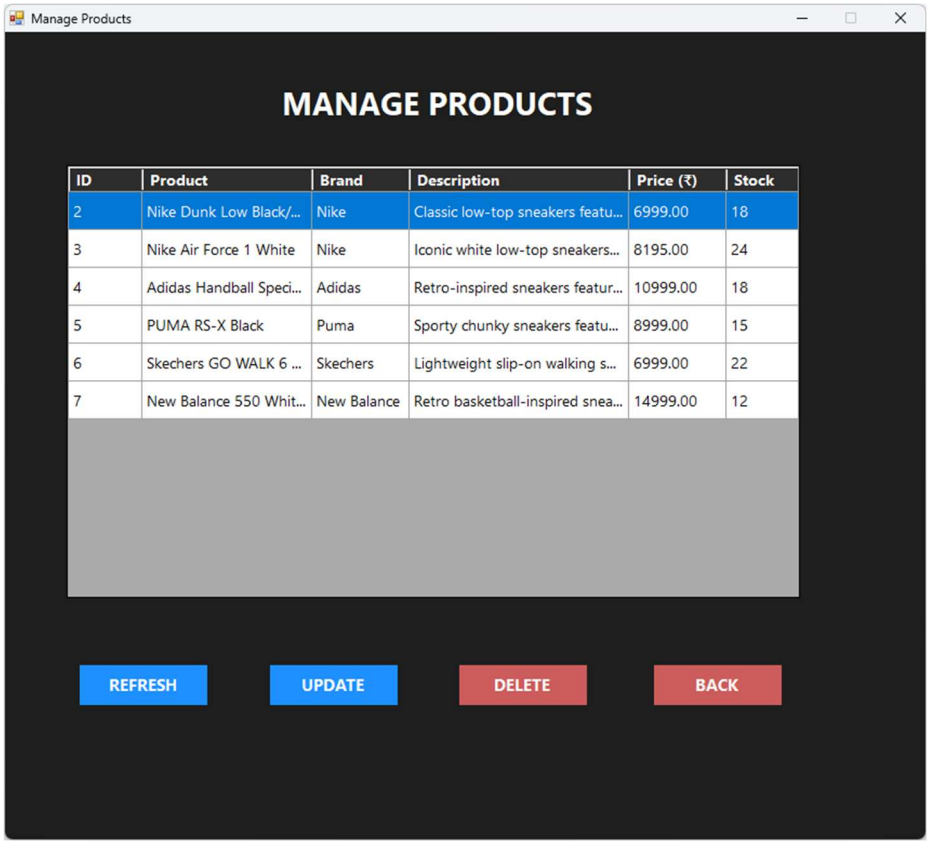


Figure 11.2— Manage Products (frmManageProducts)

View Orders

Displays order information retrieved from the Orders table and associated customer data.

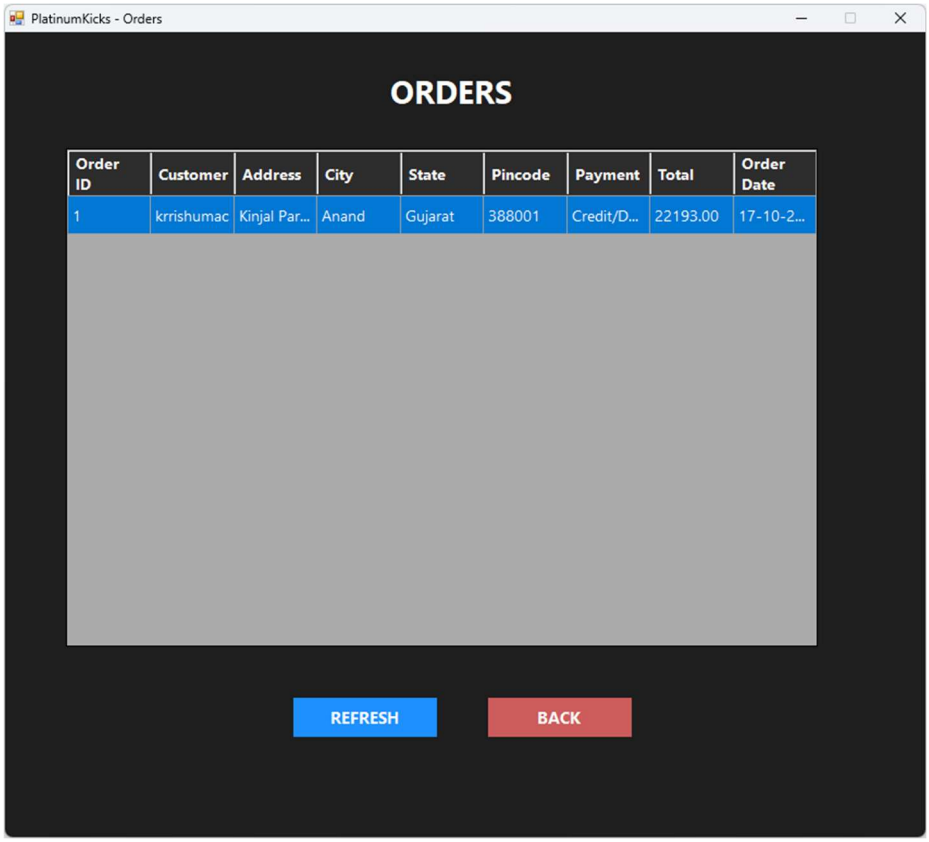


Figure 11.3 — View Orders (frmViewOrders)

Product Listing and Details

Displays products, product details, price, stock and image.

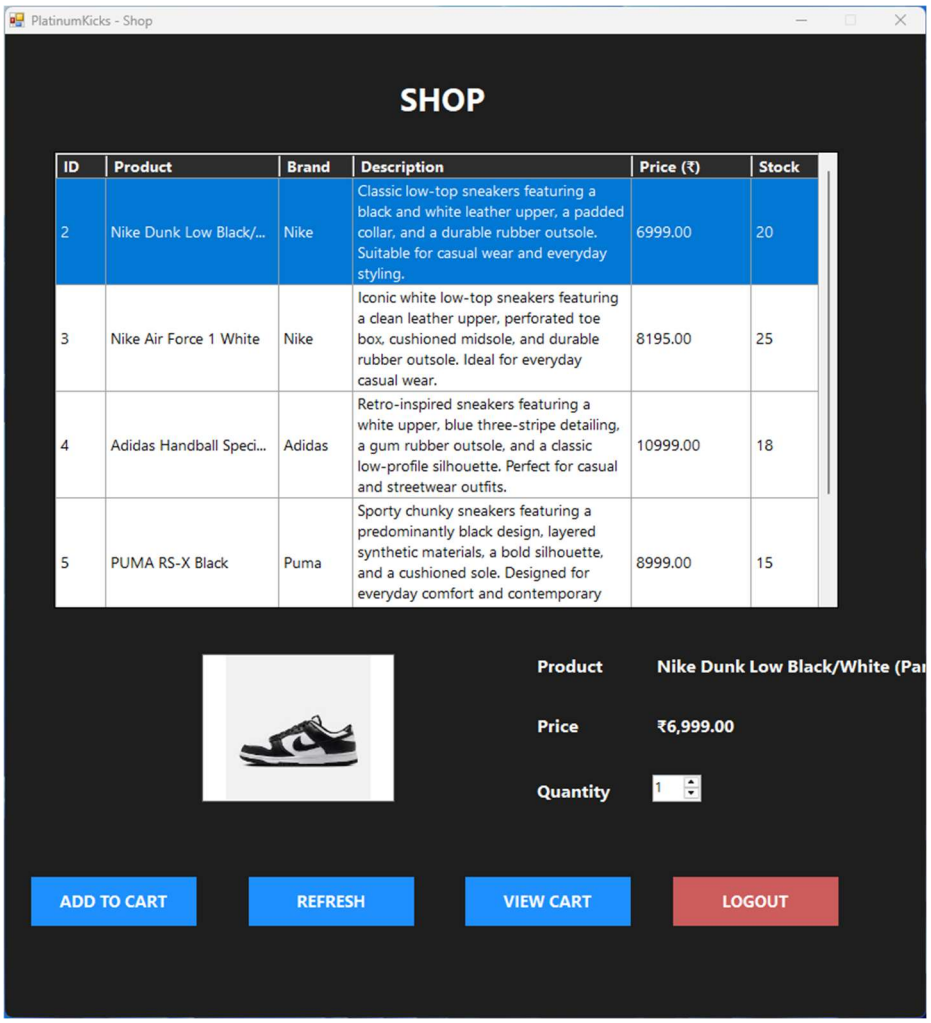


Figure 11.4— Product Listing and Details (frmShop)

Cart Summary and Checkout Result

Shows the selected cart contents and total before the order is submitted.

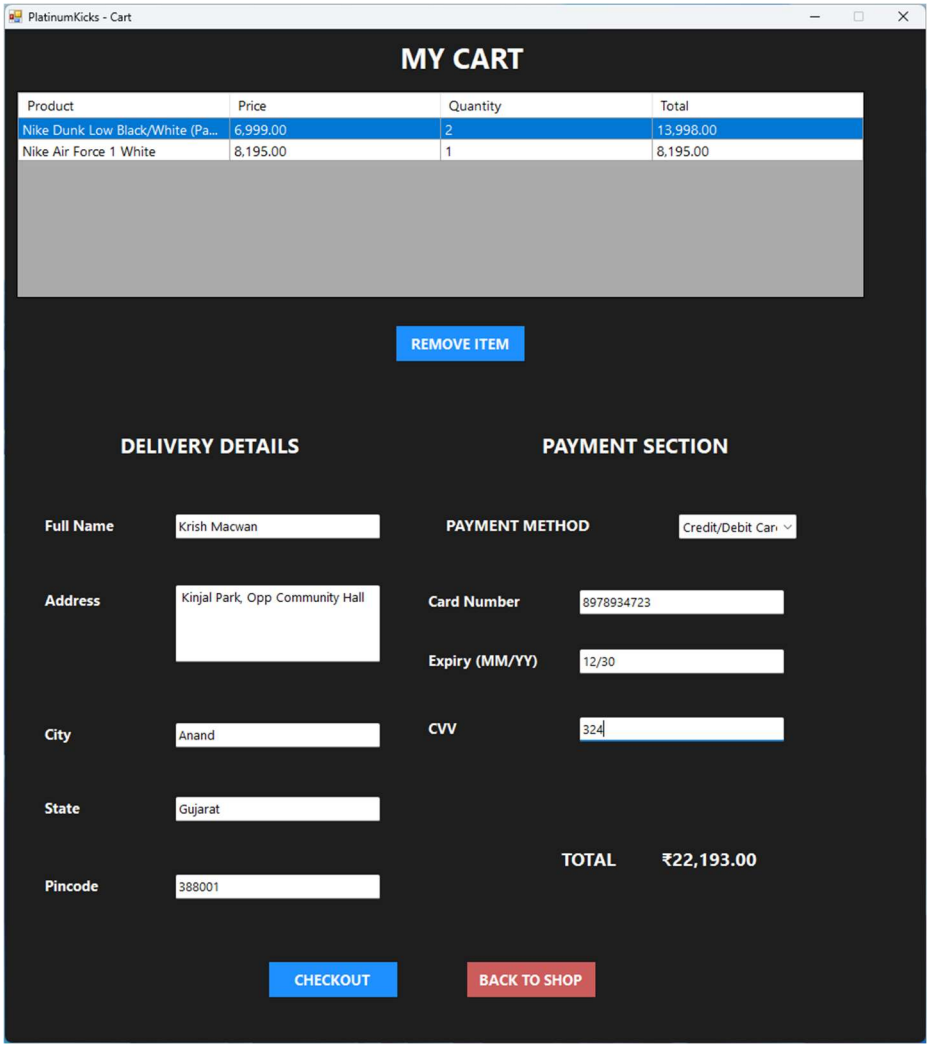


Figure 11.5— Cart Summary and Checkout Result (frmCart)

12. TESTING PROCEDURES & IMPLEMENTATION PHASES

12.1 Testing Approach

Testing was performed as functional testing of the completed desktop application. The focus was to verify that forms accept input, database operations execute, records are displayed, and the customer order flow is connected to stock updates.

12.2 Functional Test Cases

Test ID	Test operation	Expected / observed result	Status note
TC-01	Open landing screen	Application opens and displays the landing screen.	Pass — user reported completed testing
TC-02	Register a new user	Valid registration data is submitted.	Pass — user reported completed testing
TC-03	Login with valid credentials	Credentials are checked and role navigation is applied.	Pass — user reported completed testing
TC-04	Browse products	Product list and product details are displayed.	Pass after shop-selection fix
TC-05	Add products to cart	Selected products and quantities appear in cart.	Pass after cart-refresh fix
TC-06	Checkout an order	Order and order-item records are saved; stock is reduced when valid.	Tested as part of completed project workflow
TC-07	Add product as Admin	Product details and image are saved.	Implemented and tested
TC-08	Update/delete product	Admin can perform product maintenance subject to database constraints.	Delete behavior depends on current FK rule
TC-09	View orders	Order records are displayed in the order grid.	Implemented and tested
TC-10	Reopen application	Persistent records remain available when the same MDF is used.	Database-copy/location configuration must remain consistent

12.3 Implementation Phases

1. Phase 1 — Requirements and planning: identify User/Admin roles, required forms and core footwear-shopping operations.
2. Phase 2 — Interface development: create the landing, login, registration, shop, cart, admin dashboard, product-management and order-viewing forms.
3. Phase 3 — Database design: create the users, products, orders and order_items tables and their key relationships.
4. Phase 4 — Database integration: centralize SQL Server connection and commands in DatabaseHelper and connect the forms to database operations.
5. Phase 5 — Business logic: implement session handling, cart operations, order creation and stock deduction.
6. Phase 6 — Debugging and compatibility: resolve database-provider differences, form events, and cart refresh behavior.
7. Phase 7 — Functional testing: verify registration/login, product maintenance, browsing, cart, checkout and order viewing on the intended PC.
8. Phase 8 — Documentation: prepare the diagrams, database layout, screen descriptions, test cases and project report.

13. LIMITATIONS

- Payment methods are simulated; no real transaction is sent to a bank, UPI provider or card processor.
- The application is a Windows desktop application and is not available as a public web storefront.
- The shopping cart is held in application memory and is not persisted in a database table; its contents are not designed to survive closing the application.
- Order tracking, courier integration and delivery notifications are not implemented.
- The current database schema does not include separate payment transaction, shipment, category or audit-log tables.
- The application depends on the correct local SQL Server Express configuration and on the application attaching the intended MDF file.
- User interface behavior and data access are designed for the project scope and have not been described as production-scale, multi-user infrastructure.

These limitations define the current implementation scope and do not prevent the application from demonstrating the principal customer and administrator workflows.

14. CONCLUSION

Platinum Kicks - Shoes Ecommerce is a completed VB.NET Windows Forms project that demonstrates a database-backed footwear shopping workflow. It combines customer registration and login, product browsing, in-memory cart management, simulated checkout, order storage and stock updates with administrator-side product maintenance and order viewing.

The project uses Microsoft SQL Server Express and an MDF database containing Users, Products, Orders and Order Items. A centralized database helper connects the forms to persistent records, while Session and CartManager support application state during use.

The completed system meets its intended academic scope by demonstrating form-based user interaction, relational data storage, CRUD operations and order processing. Future development could introduce a real payment gateway, stronger account-security practices, persistent carts, shipping workflows and a web-based client if the project requirements expand.

Project Summary

Area	Implemented solution
Interface	VB.NET Windows Forms
Database	SQL Server Express MDF
Persistent data	Users, Products, Orders, Order Items
Customer operations	Registration, login, product browsing, cart, checkout
Admin operations	Product CRUD and order viewing
Checkout	Simulated payment method, delivery details, order creation and stock update